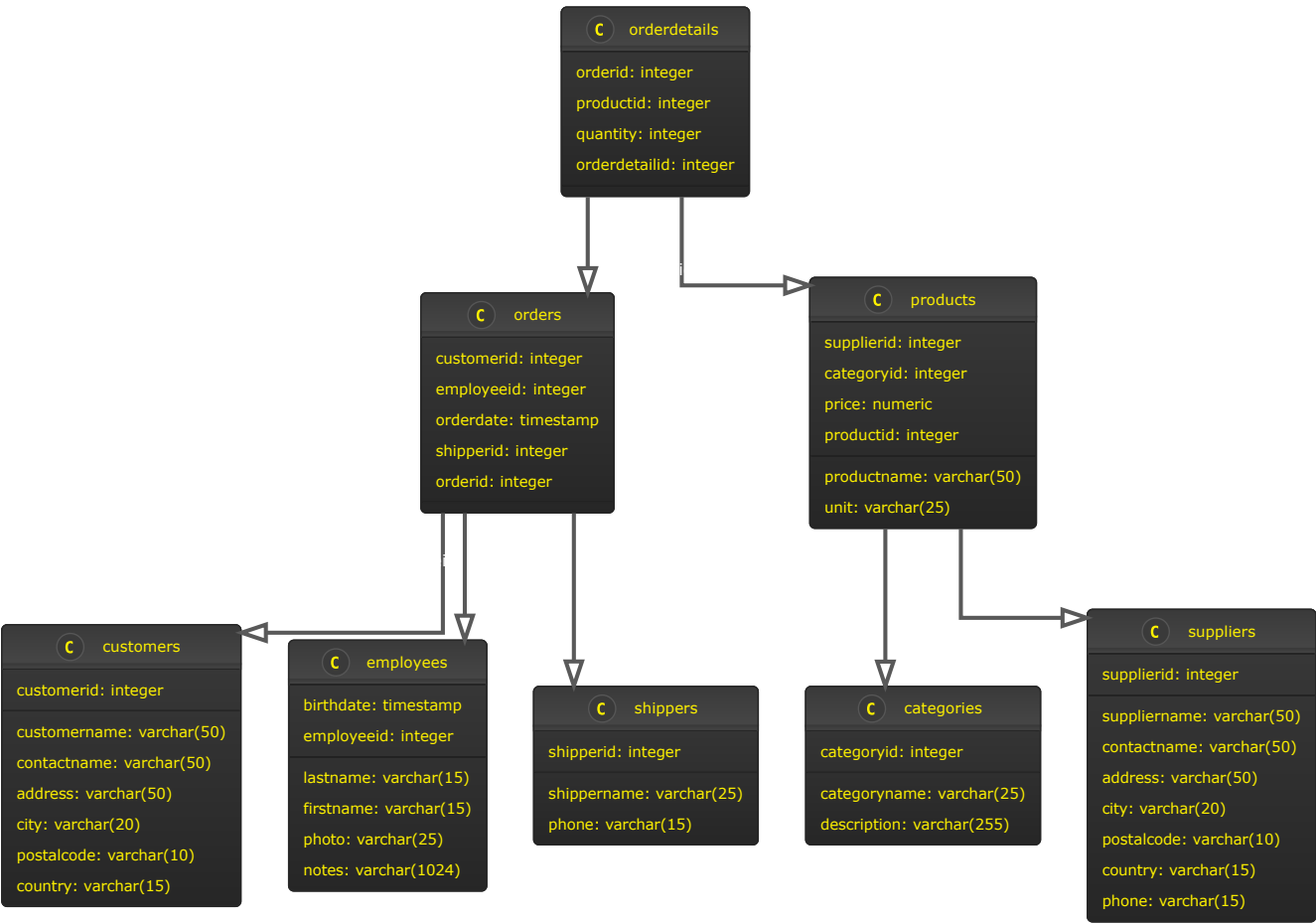


Our DB Schema - Northwind



0. Mental model first

A SQL query is basically:

1. From where do we get rows? (FROM)
2. Which rows do we keep? (WHERE)
3. How do we group them? (GROUP BY)
4. Which groups do we keep? (HAVING)
5. What do we show? (SELECT)
6. In what order? (ORDER BY)
7. How many? (LIMIT)

Order of execution:

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

1. Look at the table

Before querying, know what's inside.

Important are: *table names, column names, data types*

```
SELECT *  
FROM Customers  
LIMIT 10;
```

You'll see columns like CustomerID, CompanyName, ContactName, Country, etc.

2. The absolute core query

```
SELECT CustomerID, CompanyName  
FROM Customers;
```

Or (lazy mode):

```
SELECT *  
FROM Products;
```

3. Filtering rows – WHERE

```
SELECT ProductName, UnitPrice  
FROM Products  
WHERE UnitPrice > 50;
```

Common operators:

- = equals
- <> or != not equal

- <, >, <=, >=
- BETWEEN 10 AND 20
- IN (...)
- LIKE 'U%' (pattern) alle die mit großem U beginnen
- IS NULL / IS NOT NULL

```
SELECT *
FROM Customers
WHERE Country IN ('USA', 'UK', 'Germany')
AND City LIKE 'L%';
```

4. Sorting – ORDER BY

```
SELECT ProductName, UnitPrice
FROM Products
ORDER BY UnitPrice DESC;
```

You can order by multiple:

```
SELECT ProductName, CategoryID, UnitPrice
FROM Products
ORDER BY CategoryID ASC, UnitPrice DESC;
```

- ASC = ascending (default)
- DESC = descending

You can order by columns you didn't select (Postgres allows it).

5. Limiting – LIMIT

```
SELECT *
FROM Orders
ORDER BY OrderDate DESC
LIMIT 10;    LIMIT 10 = wir wollen nur 10 Einträge haben
```

Handy for testing.

6. Calculations – expressions & aliases

You can compute stuff in SELECT:

```
SELECT ProductName,
       UnitPrice * UnitsInStock AS TotalStockValue
FROM Products;
```

- || = string concat in Postgres
- AS gives the column a nicer name

7. Aggregations – COUNT, SUM, ...

Without grouping (whole table is one group):

```
SELECT COUNT(*) AS TotalCustomers * = immer alles COUNT(*) = es soll immer alle Einträge zählen  
FROM Customers;                (nimm alle und zähl sie durch)
```

Or:

```
SELECT AVG(UnitPrice) AS AveragePrice  
FROM Products;
```

Other fns: SUM(), MIN(), MAX()

8. GROUP BY

Rule: every column in SELECT must be either *aggregated* or in the GROUP BY

Example: "How many products per category?"

```
SELECT CategoryID,  
       COUNT(*) AS ProductCount  
FROM Products  
GROUP BY CategoryID  
ORDER BY ProductCount DESC;
```

That says: take all rows, group them by CategoryID, and for each group count.

Do I have to use an *aggregate* with GROUP BY?

Yes. GROUP BY without any *aggregate* is pointless.

9. HAVING

- WHERE filters rows before grouping.
- HAVING filters groups after grouping.

Example: "Show only categories with more than 10 products."

```
SELECT CategoryID,  
       COUNT(*) AS ProductCount  
FROM Products  
GROUP BY CategoryID  
HAVING COUNT(*) > 10;
```

10. Joins – connecting tables

INNER JOIN

Get each product's supplier name:

```
SELECT p.ProductName,  
       s.CompanyName AS Supplier  
FROM Products AS p  
JOIN Suppliers AS s  
  ON p.SupplierID = s.SupplierID;
```

JOIN = **INNER JOIN** = keep only matches in both tables.

LEFT JOIN

Show all products, even if they have no supplier (theoretically):

```
SELECT p.ProductName,  
       s.CompanyName AS Supplier  
FROM Products AS p  
LEFT JOIN Suppliers AS s  
  ON p.SupplierID = s.SupplierID;
```

Rows without match → **NULL** on right side.

11. Join + filtering

Join first, then filter.

```
SELECT o.OrderID,  
       c.CompanyName AS Customer,  
       o.OrderDate  
FROM Orders o  
JOIN Customers c ON o.CustomerID = c.CustomerID  
WHERE c.Country = 'Germany';
```

12. Join + group by

Example: "How many orders per customer?"

```
SELECT c.CompanyName,  
       COUNT(o.OrderID) AS OrdersCount  
FROM Customers c  
JOIN Orders o ON c.CustomerID = o.CustomerID  
GROUP BY c.CompanyName  
ORDER BY OrdersCount DESC;
```

13. Subqueries

As a table

```
SELECT *
FROM (
    SELECT ProductID, SUM(Quantity) AS TotalSold
    FROM OrderDetails
    GROUP BY ProductID
) AS Sales
WHERE TotalSold > 500;
```

In WHERE

“Customers who ordered from USA suppliers”

```
SELECT DISTINCT c.CompanyName
FROM Customers c
WHERE c.CustomerID IN (
    SELECT o.CustomerID
    FROM Orders o
    JOIN [Order Details] od ON o.OrderID = od.OrderID
    JOIN Products p ON od.ProductID = p.ProductID
    JOIN Suppliers s ON p.SupplierID = s.SupplierID
    WHERE s.Country = 'USA'
);
```

14. CRUD basics

INSERT

```
INSERT INTO Categories (CategoryName, Description)
VALUES ('Luxury Goods', 'High-end specialty items');
```

Multiple:

```
INSERT INTO Shippers (CompanyName, Phone)
VALUES
    ('Dragon Express', '(555) 777-7777'),
    ('Polar Freight', '(555) 222-2222');
```

UPDATE

```
UPDATE Products
SET UnitPrice = UnitPrice * 1.10
WHERE CategoryID = 1;
```

⚠ Always have a WHERE, unless you mean to update everything.

DELETE

```
DELETE FROM Employees
WHERE LastName = 'Davolio';
```

⚠ Again: WHERE or you delete the universe.

15. Helpful extras

Distinct

```
SELECT DISTINCT Country
FROM Customers;
```

Distinct -> doppelte werden aussortiert

Case

```
SELECT ProductName,
       CASE CASE ist wie ein "If"
         WHEN UnitPrice < 20 THEN 'Cheap'
         WHEN UnitPrice BETWEEN 20 AND 50 THEN 'Moderate'
         ELSE 'Expensive'
       END AS PriceCategory
FROM Products;
```

Date stuff

```
SELECT OrderID,
       EXTRACT(YEAR FROM OrderDate) AS OrderYear
FROM Orders;
```

16. How to think - Translating thought to SQL

1. Say it in German/English first:

"I want all German customers and their order dates, sorted by newest first."

2. Identify tables: Customers, Orders

3. Identify join condition: Customer.CustomerID = Orders.CustomerID

4. Identify filters: Customer.Country = 'Germany'

5. Identify output columns: CompanyName, OrderDate

6. Add order.

```
SELECT c.CompanyName, o.OrderDate
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE c.Country = 'Germany'
ORDER BY o.OrderDate DESC;
```

17. Common mistakes

- "column must appear in the GROUP BY" → either *aggregate* it or include it
- WHERE COUNT(...) → use HAVING

- `= NULL` → use `IS NULL`
 - forgetting `ON` in joins
 - blindly `SELECT *` in multi-join → chaos
-

18. Handy patterns

Top N expensive products

```
SELECT ProductName, UnitPrice
FROM Products
ORDER BY UnitPrice DESC
LIMIT 5;
```

Total sales per day

```
SELECT DATE(o.OrderDate) AS Day,
       SUM(od.Quantity * od.UnitPrice) AS Sales
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY DATE(o.OrderDate)
ORDER BY Day;
```

Customers with no orders

```
SELECT c.CustomerID, c.CompanyName
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```